1. 装饰器的概念及简单应用

1.1 闭包函数

嵌套函数：如果在一个函数的内部还定义了另一个函数，这个函数就叫嵌套函数。外部的我们叫它外函数，内部的我们叫他内函数。

闭包函数：如果内函数使用了外函数的局部变量，并且外函数把内函数返回出来的过程叫做闭包，里面的内函数是闭包函数。

```
def nth_power(exponent):
    def exponent_of(base):
        return base ** exponent
    return exponent_of

square = nth_power(2)
cube = nth_power(3)

print(square(2))
print(cube(2))
```

1.2 装饰器的概念

在Python中，装饰器（Decorator）本质上是一种特殊的**嵌套函数**，它接收一个函数作为参数（该函数就是被装饰的函数），并返回一个新的函数（装饰之后的函数）。装饰器最大的作用就是可以让我们在不改变被装饰函数的代码的情况下去给它添加新的功能。

1.3 装饰器的基本用法

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
def func1(func):
    def inner():
        print('b')
        func()
        print('c')
    return inner

def func():
    print('a')

func = func1(func)
func()
```

其中，func()函数原本的功能是打印字符串a，但是将其作为参数传给func1并将结果赋值给func之后，再次执行func的结果就是先打印字符串b，再打印字符串a，然后打印字符串c。

在这个过程中，原函数func的代码并没有被修改，而最后执行func()的结果确是和之前大不相同了，原因就在于func作为了参数传递给了func1，并且在func1函数中添加了新的内容，且最后的返回值又重新赋值给了func，也就是说，此时的func函数以及不是原先的打印字符串a的函数了，而是func1函数所返回的inner函数，那么下面的func()实际上等同于inner()。那么这段代码本质上是修改了调用的函数，但是并没有修改调用方式，并且还实现了附加的功能。

如果将func作为参数传递给func1而返回值并没有重新赋值给func的话，那么它就是一个闭包函数。但是如果返回值重新赋值给func的话，那么func1函数就是一个装饰器函数，它将对接收到的func参数添加打印字符串b和c的功能，且不影响func原有的功能。

华清远见开源实验中心 yyzlab.com.cn

1.4 装饰器语法糖

语法糖：语法糖是指一种编程语言提供的，可以让代码更加简洁、易于理解和书写的语法特性，这些特性不会带来新的功能，也就是说，使用或不使用语法糖，最终程序的行为是相同的，但是语法糖可以使代码更易于阅读和维护。

Python中有许多语法糖，比如：

- **列表推导式**
- **字典推导式**
- **集合推导式**

华清远见 HQYJ.COM

- **链式比较**
- **f-string**
- **解包**
- **装饰器**

在Python中，装饰器的使用可通过在目标函数上方放置一个 `@` 符号，随后跟上装饰器函数的名字来实现。这样，我们就无需显式地将目标函数作为参数传递给装饰器函数，而是可以直接调用目标函数。通过这种方式，装饰器能够为原有函数增添额外的功能。比如上面的例子可以改为：

```
def func1(func):
    def inner():
        print('b')
        func()
        print('c')
    return inner


@func1
def func():
    print('a')


func()
```

1.5 装饰带参数的函数

当装饰器装饰带有参数的函数时，装饰器内部也需要去接收参数，从而保证在装饰器里调用被装饰的函数时能够正常传递参数。

1.5.1 装饰只有一个参数的函数

```
import time

def timer(f):

    def decorator(x):
        start = time.time()
        ret = f(x)
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
        print(time.time() - start)
        return ret
    return decorator

@timer
def my_func(x):
    time.sleep(x)

my_func(1)
```

1.5.2 装饰未知参数数量的函数

```
import time

def timer(f):

    def decorator(*args, **kwargs):
        start = time.time()
        ret = f(*args, **kwargs)
        print(time.time() - start)
        return ret
    return decorator


@timer
def my_func(x):
    time.sleep(x)

@timer
def add(x, y):
    time.sleep(1)
    print(x + y)

my_func(1)
add(2, 3)
```

华清远见|元宇宙实验中心 yyzlab.com.cn

1.6 带参数的装饰器

华清远见 HQYJ.COM

上面的装饰带参数的函数是指被装饰的函数带参数，而这里的带参数的装饰器是指装饰器本身需要参数。而装饰器本身又要接收被装饰的函数名称，并且返回一个新的函数，所以为了保证装饰器能够顺利接收参数及顺利接收被装饰的函数，需要在装饰器内部多一层内嵌函数，最外层的函数用来获取装饰器参数，中间的函数用来获取被装饰的函数名称，最内层的函数负责进行功能的添加。

```
def func1(prefix):
    def decorator(func):
        def wrapper(*args, **kwargs):
            result = func(*args, **kwargs)
            return f"{result} {prefix}"
        return wrapper
    return decorator

# 使用装饰器并传递参数
@func1("How are you?")
def greet(name):
    return f"Hello, {name}."

# greet = func1('How are you?')(greet)

# 调用被装饰的函数
print(greet("Alice"))
```

1.7 装饰器嵌套

装饰器嵌套是指在Python中，一个函数可以被多个装饰器依次装饰。每个装饰器都会对原始函数进行一层包装，从而在调用原始函数时，可以依次执行每个装饰器中的逻辑，以达到为该函数添加多个不同功能的要求。

```
def decorator_one(func):
    def wrapper_one(*args, **kwargs):
        print("Decorator one - before call")
        result = func(*args, **kwargs)
        print("Decorator one - after call")
        return result
    return wrapper_one

def decorator_two(func):
    def wrapper_two(*args, **kwargs):
        print("Decorator two - before call")
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
        result = func(*args, **kwargs)
        print("Decorator two - after call")
        return result
    return wrapper_two

@decorator_one
@decorator_two
def say_hello(name):
    print(f"Hello, {name}")

# say_hello = decorator_one(decorator_two(say_hello))
say_hello("Alice")
```

装饰器嵌套时，装饰器的执行顺序是从下到上，且最下面的装饰器的返回值会返回给上一层的装饰器作为参数，然后层层传递直到最上面的装饰器，而当我们要运行被装饰的函数时，相当于从最上层的装饰器开始向下运行直到运行完毕。

1.8 类装饰器

除了可以自定义一个新的函数用作装饰器之外，也可以将一个类作为装饰器，为被装饰的函数添加新的功能。类装饰器通过实现类的__call__方法，使得类的实例可以被当作函数来调用，从而实现对其他函数的装饰。

```
class MyDecorator:
    def __init__(self, func):
        self.func = func


    def __call__(self, *args, **kwargs):
        print("Something is happening before the function is
called.")
        result = self.func(*args, **kwargs)
        print("Something is happening after the function is
called.")
        return result

@MyDecorator
def say_hello(name):
    print(f"Hello, {name}")

say_hello("Alice")
```

华清远见|云计算实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

2. 装饰器的常见应用

2.1 记录日志

装饰器可以用来记录函数调用的详细信息，包括调用时间、参数、返回值等。

```
import logging

def log_decorator(func):
    def wrapper(*args, **kwargs):
        logging.basicConfig(filename='./app.log',
level=logging.INFO, filemode='a', format='%(name)s - %(levelname)s
- %(asctime)s- %(message)s')
        logging.warning(f"Calling function: {func.__name__} with
args: {args} and kwargs: {kwargs}")
        result = func(*args, **kwargs)
        logging.warning(f"Function {func.__name__} returned:
{result}")
        return result
    return wrapper


@log_decorator
def test():
    print('123')

test()
```

2.2 性能检测

装饰器可以用来测量函数执行所需的时间，帮助识别性能瓶颈。

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        print(f"{func.__name__} took {end_time - start_time:.10f}
seconds to run.")
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
        return result
    return wrapper


@timer
def process_list(my_list):
    for i in my_list:
        print(i)


my_list = list(range(100))
process_list(my_list)
```

2.3 权限验证

装饰器可以用来检查用户是否有权限执行特定的函数或方法。

```
class User:
    def __init__(self, username, role):
        self.username = username
        self.role = role

# 装饰器：检查用户是否具有管理员权限
def admin_only(func):
    def wrapper(user, *args, **kwargs):
        if user.role != 'admin':
            raise PermissionError(f"User {user.username} is not
authorized to perform this action.")
        return func(user, *args, **kwargs)
    return wrapper

# 使用装饰器的函数
@admin_only
def delete_user(user, user_to_delete):
    """只有管理员可以删除用户"""
    print(f"User {user_to_delete.username} has been deleted by
{user.username}.")

# 测试代码
admin_user = User('Alice', 'admin')
normal_user = User('Bob', 'user')
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```python
# 尝试以管理员身份删除用户
delete_user(admin_user, normal_user)  # 应该成功

# 尝试以普通用户身份删除用户
try:
    delete_user(normal_user, admin_user)  # 应该失败
except PermissionError as e:
    print(e)
```

华清远见|元宇宙实验中心
yyzlab.com.cn